

Aula 3 — Testes de SPA & PWA

"Como testar PWA offline? Como verificar hydration?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 08/10/2026

Recap da Aula 2 — o que de fato fizemos

- Playwright por dentro: CDP, locators, auto-waiting, flakiness
- `await expect(...)` — `expect` de locator/page é assíncrono (e precisa de matcher)
- Network mocking com `page.route()` — spec 02, testes 1–4
- `storageState` / `auth.setup.ts` : login uma vez, reaproveitado nos specs

Ficou pra hoje: visual regression (spec 03) e os blocos 5–7 do spec 02 (opcionais).

| Dúvida no spec 02? Chama agora. O bot ainda aceita PR até o prazo no Canvas.

Agenda da aula (3h30)

- **90min** — SPA & PWA: SPA vs hydration, Service Worker, manifest, offline (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on: specs 03 (visual), 04 (SPA), 05 (PWA/offline) + Lighthouse CI
- **15min** — Wrap-up e entrega

Objetivos da aula

- **Testar** hydration em SPAs (Next.js App Router, RSC)
- **Testar** Service Worker lifecycle e cache strategies
- **Validar** PWAs (manifest, install prompt, offline mode)
- **Configurar** Lighthouse CI bloqueando merge
- **Aplicar** performance budgets como teste

Particularidades de SPAs

SPAs introduzem fases de execução que testes precisam respeitar:

1. **HTML inicial** (server-side render ou static)
2. **Hydration** — JS anexa event listeners ao HTML
3. **Interativo** — user pode clicar e funciona

| **Teste em fase 1** que parece passar pode falhar em produção quando user clica antes de hydration completar.

Hydration testing

```
test('button works after hydration', async ({ page }) => {  
  await page.goto('/');  
  
  // Aguardar networkidle (heurística: tudo carregado + JS provavelmente hidratou)  
  await page.waitForLoadState('networkidle');  
  
  // Para ser explícito, aguarde marker do framework  
  await page.waitForFunction(() => window.__NEXT_DATA__);  
  
  await page.getByRole('button', { name: 'Click me' }).click();  
  await expect(page.getByText('Clicked!')).toBeVisible();  
});
```

Em React Server Components, hydration acontece **por boundary** — testes precisam aguardar especificamente.

Service Worker testing — desafios

SW roda em **thread separada**, não no main thread. Testes precisam:

1. Forçar registro (não automático em dev)
2. Aguardar `activate`
3. Conhecer cache state
4. Simular network condition

Forçar SW em test env

```
test('SW intercepts API calls', async ({ page, context }) => {  
  await page.goto('/');  
  
  // Aguardar SW estar ready  
  await page.evaluate(async () => {  
    const reg = await navigator.serviceWorker.ready;  
    return reg.active?.state;  
  });  
  
  // Agora SW intercepta requests  
  await page.goto('/dashboard');  
});
```

Cache strategies — testar

```
test('cache-first serves from cache', async ({ page, context }) => {  
  // Primeira visita – popular cache  
  await page.goto('/');  
  await page.waitForLoadState('networkidle');  
  
  // Bloquear network completamente  
  await context.setOffline(true);  
  
  // Recarregar – deve servir do cache  
  await page.reload();  
  await expect(page.getByText('Welcome')).toBeVisible();  
});
```

SW lifecycle — skipWaiting test

```
test('SW updates when new version deployed', async ({ page }) => {  
  // Cenário: nova versão SW disponível  
  await page.goto('/');  
  
  await page.evaluate(() => {  
    return new Promise<void>((resolve) => {  
      navigator.serviceWorker.addEventListener('controllerchange', () => resolve());  
      navigator.serviceWorker.ready.then((reg) => {  
        // Simula update  
        reg.update();  
      });  
    });  
  });  
  
  // Verificar que cliente foi assumido por novo SW  
});
```

Cenários intermediários (waiting state) são onde bugs aparecem.

PWA — manifest validation

```
test('manifest is valid', async ({ page }) => {  
  const response = await page.goto('/manifest.json');  
  const manifest = await response?.json();  
  
  expect(manifest.name).toBeTruthy();  
  expect(manifest.short_name).toBeTruthy();  
  expect(manifest.start_url).toBe('/');  
  expect(manifest.display).toBe('standalone');  
  expect(manifest.icons.length).toBeGreaterThan(0);  
  expect(manifest.icons.find((i) => i.sizes === '192x192')).toBeDefined();  
  expect(manifest.icons.find((i) => i.sizes === '512x512')).toBeDefined();  
});
```

PWA — install prompt simulation

```
test('install prompt fires on supported browsers', async ({ page }) => {
  let installPromptFired = false;

  await page.exposeFunction('onBeforeInstallPrompt', () => {
    installPromptFired = true;
  });

  await page.addInitScript(() => {
    window.addEventListener('beforeinstallprompt', () => {
      (window as any).onBeforeInstallPrompt();
    });
  });

  await page.goto('/');
  // Trigger criteria do browser

  expect(installPromptFired).toBe(true);
});
```

PWA — offline mode test

```
test('catálogo continua acessível offline', async ({ page, context }) => {  
  await page.goto('/qa');  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  
  // SW "activated" NÃO basta: precisa CONTROLAR a página  
  await page.waitForFunction(() => navigator.serviceWorker.controller !== null);  
  
  await context.setOffline(true); // perda total de rede  
  await page.reload();  
  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  await expect(page.getByTestId('offline-banner')).toBeVisible();  
});
```

Sequência: visita online → SW controla → offline → reload. Pular o 2º passo é o erro mais comum.

Background Sync — extra (fora do lab)

```
test('queues message for sync when offline', async ({ page, context }) => {
  await page.goto('/messages');
  await context.setOffline(true);

  await page.fill('[name=message]', 'Olá!');
  await page.click('button[type=submit]');

  // Mensagem fica em estado "pending sync"
  await expect(page.getByText('Pending sync')).toBeVisible();

  // Voltar online
  await context.setOffline(false);

  // Background Sync deve disparar
  await page.evaluate(async () => {
    const reg = await navigator.serviceWorker.ready;
    return reg.sync.register('flush-outbox');
  });

  await expect(page.getByText('Sent')).toBeVisible({ timeout: 5000 }); // auto-wait, sem sleep
});
```

Lighthouse CI — visão geral

```
npm install -g @lhci/cli@0.13.x
```

```
# rodar local  
lhci autorun
```

```
// lighthouserc.json  
{  
  "ci": {  
    "collect": {  
      "url": ["http://localhost:3000/"],  
      "numberOfRuns": 3  
    },  
    "assert": {  
      "assertions": {  
        "categories:performance": ["error", { "minScore": 0.9 }],  
        "categories:pwa": ["error", { "minScore": 0.9 }],  
        "categories:accessibility": ["error", { "minScore": 0.9 }],  
        "first-contentful-paint": ["warn", { "maxNumericValue": 2000 }],  
        "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }]  
      }  
    }  
  }  
}
```

Performance budgets

Conceito de **Tim Kadlec (2013)** popularizado por web.dev. No lab, os budgets estão no `lighthouse.json`:

```
"assertions": {  
  "categories:performance": ["error", { "minScore": 0.9 }],  
  "categories:accessibility": ["error", { "minScore": 0.9 }],  
  "categories:best-practices": ["error", { "minScore": 0.9 }],  
  "first-contentful-paint": ["error", { "maxNumericValue": 2000 }],  
  "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }],  
  "total-blocking-time": ["error", { "maxNumericValue": 300 }]  
}
```

Também dá pra limitar tamanho de bundle (`resource-summary:script:size`). **Death by a thousand cuts** evitado.

Lighthouse CI no GitHub Actions

```
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
    env:
      LHCI_GITHUB_APP_TOKEN: ${ secrets.LHCI_GITHUB_APP_TOKEN }
```

Resultado posta comment no PR com link pra report. Falha bloqueia merge.

IA aplicada à análise Lighthouse

Em vez de só ler números, **alimentar relatório pra LLM**:

```
Você é especialista em performance web. Analise este Lighthouse report:
```

```
[JSON do report]
```

```
Identifique:
```

1. Quais regressões são significativas (semanticamente)?
2. Quais são triviais (variação de medição)?
3. Quais ações concretas recomendaria?

```
Responda em formato estruturado.
```

IA faz **interpretação semântica**, não só comparação numérica.

Cobertura de código — o que o relatório mede

Cobertura = **quanto do código-fonte foi executado** pelos testes. Quatro métricas:

Métrica	Pergunta que responde
Statements	quantas instruções rodaram?
Branches	os dois lados de cada <code>if / ?:</code> rodaram?
Functions	cada função foi chamada ao menos 1 vez?
Lines	quantas linhas rodaram?

No e2e do CineFav: cobertura **V8 do Chromium** (`page.coverage.startJSCoverage()`) + sourcemap → relatório por arquivo de `src/`.

Relatório real — suíte e2e do CineFav

All files

78.47% Statements 113/144 67.39% Branches 62/92 58.82% Functions 60/102 77.85% Lines 109/140

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
App.tsx	100%	7/7	100%	0/0
components	75%	15/20	58.33%	7/12
main.tsx	100%	1/1	100%	0/0
screens	84%	42/50	74.46%	35/47
services	72.72%	24/33	47.36%	9/19
store	71.87%	23/32	78.57%	11/14
utils	100%	1/1	100%	0/0

Code coverage generated by Istanbul at 2026-10-03T19:01:00.670Z

78% statements · 67% branches · 59% functions. No vermelho: services (branches 47%) e utils (functions 44%).

Dentro de um arquivo — o que ficou de fora

All files / store favorites.ts

57.89% Statements 11/19 62.5% Branches 5/8 55.55% Functions 5/9 55.55% Lines 10/18

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1 // src/store/favorites.ts
2 //
3 // Favoritos persistidos em localStorage, com useSyncExternalStore pra
4 // qualquer tela reagir a mudanças (mesma semântica do favoritesStore mobile).
5
6 import { useSyncExternalStore } from 'react';
7
8 26x const KEY = 'cinefav-favorites';
9
10 type Listener = () => void;
11 const listeners = new Set<Listener>();
12
13 13x function read(): number[] {
14 13x   const raw = localStorage.getItem(KEY);
15 13x   if (!raw) return [];
16   try {
17     return JSON.parse(raw) as number[];
18   } catch {
19     return [];
20   }
21 }
22
23 26x let snapshot: number[] = [];
24 let snapshotRaw = '';
25
26 277x function getSnapshot(): number[] {
27 277x   const raw = localStorage.getItem(KEY) ?? '[]';
28 277x   if (raw !== snapshotRaw) {
29     snapshotRaw = raw;
30     snapshot = read();
31   }
32   return snapshot;
33 }
34
```

Verde = executado (13x , 277x). Vermelho = nunca executado: o `try/catch` do `JSON.parse` em `store/favorites.ts` .

Cobertura ≠ qualidade

- Linha executada **não** é comportamento verificado: um teste **sem** `await` / `matcher` executa o código e dá 100% de cobertura sem checar nada
- Use o relatório pra achar o que **não** foi testado (o vermelho), não como meta de nota
- Gate sensato no CI: piso por **branches** (ex.: 70%), não 100%

Na Aula 6: por que **mutation testing** prova o que a cobertura só sugere.

Lab — specs 03, 04 e 05 + Lighthouse (90min)

```
cd exercicios/01-lab-web-pwa-playwright/pratica · ls tests/e2e
```

1. **Spec 03 — visual regression** (25min): `toHaveScreenshot` , 3 viewports, máscara. Teoria nos slides da Aula 2
2. **Spec 04 — SPA** (20min): `data-app-ready` , navegação client-side, lazy chunk
3. **Spec 05 — PWA/offline** (30min): SW ativo, manifest, offline com controller
4. **Lighthouse CI** (15min): `lighthouseci.json` + 3 budgets rodando

Tudo em `/qa` (ambiente determinístico), contra o **build** (`vite preview`) — SW não existe no dev server.

Pitfalls comuns

- ✗ Testar SPA sem aguardar hydration — falha intermitente
- ✗ Reset de cache Service Worker entre tests — esquecer = test pollution
- ✗ Lighthouse CI rodando uma vez só — variação alta, configure 3+ runs
- ✗ Budget muito apertado — bloqueia PRs legítimos por 10ms
- ✗ Não monitorar Web Vitals em produção — divergência entre lab e prod
- ✗ Offline sem esperar o SW controlar a página — `activated` não basta, falta `controller`
- ✗ `expect` de locator sem `await` — verde sem checar, ou erro "page closed"

Atividade — Lab Web + PWA (20 pts, fecha nesta aula)

Entrega: fork + PR. Prazo: ver Canvas. Enunciado completo no repo.

#	Critério	Pontos
1	Auth state reuse (<code>storageState</code>)	3
2	Network mocking (<code>route()</code>)	3
3	Visual regression, 3 viewports	4
4	SW lifecycle testado	3
5	Offline mode (<code>setOffline</code>)	3
6	Lighthouse CI, 3 budgets	4

Eliminatório: suíte 100% verde em 3 runs consecutivos.

Leituras obrigatórias

- Archibald, J. — *Service Workers Testing Strategies* (web.dev)
- web.dev — *Lighthouse CI* (oficial)
- Lock, A. — *Testing Service Workers* (artigo)
- **Mobile.dev Docs** — preparação Aula 4

Próxima aula — Aula 4

Automação Mobile com Maestro

- YAML declarativo
- Conditional flows + runFlow
- Maestro Studio
- Comparativo Maestro vs Appium vs Detox
- Cloud devices

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith